

Cahier des charges d'un système de Gestion des Rendez- vous Médicaux :

- **Nom de l'établissement :** EST LAÂYOUNE.
- **Module:** CONCEPTION UML.
- **Titre du projet :** SYSTEME DE GESTION DE RENDEZ-VOUS MEDICAUX.
- **Réalisé par :** ETTAIB HIND.
- **Filière :** CONCEPTION ET DEVEMOPEMENT DES LOGICIELS.

I. INTRODUCTION:

- **1.1 Contexte:**

- Ce document constitue le cahier des charges pour la conception et la modélisation d'un Système de Gestion des Rendez-vous Médicaux. Ce projet est réalisé dans le cadre du module UML pour l'année 2025-2026.

- **1.2 Objectif du système:**

- L'objectif est de développer un système logiciel permettant de gérer efficacement la prise de rendez-vous dans un cabinet médical ou une clinique. Le système doit faciliter la gestion des patients et des médecins, centraliser l'historique médical des patients, et automatiser les notifications.

II. IDENTIFICATION DES ACTEURS

ET BESOINS:

2.1 Acteurs du système:

- **Patient:** Personne souhaitant consulter un médecin. Peut prendre, annuler ou consulter ses rendez-vous et son historique médical.
- **Médecin:** Professionnel de santé. Consulte son agenda, valide des rendez-vous, accède à l'historique médical.
- **Secrétaire:** Personnel administratif. Gère les patients, les médecins, et les agendas. Prend des rendez-vous pour le compte des patients.
- **Système de Notification:** Acteur secondaire externe (API email/SMS) déclenché pour envoyer des confirmations et rappels.

2.2 Besoins fonctionnels:

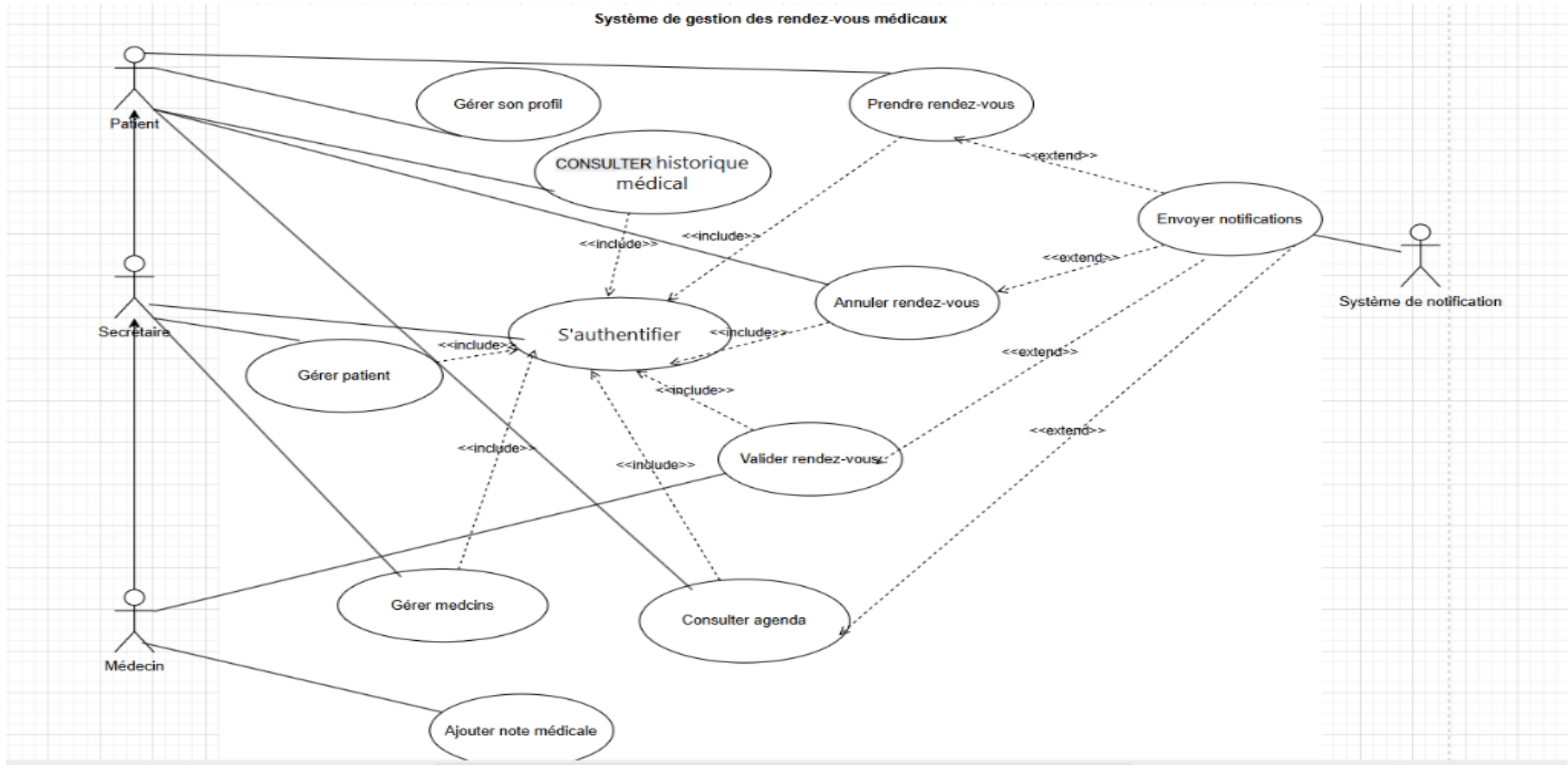
- **Gestion des patients:**Le secrétaire peut enregistrer un nouveau patient dans le système en saisissant ses informations personnelles (nom, prénom, date de naissance, adresse, téléphone, email, numéro de sécurité sociale, mutuelle).
- **Gestion des médecins:**Le secrétaire peut ajouter un nouveau médecin dans le système avec ses informations : nom, prénom, spécialité, numéro RPPS, tarif de consultation, horaires de travail.
- **Prise de rendez-vous:**Un patient ou un secrétaire peut prendre un rendez-vous en sélectionnant un médecin, une date et un créneau horaire disponible. Le système vérifie la disponibilité avant de confirmer.
- **Annulation de rendez-vous:**Un patient, un médecin ou un secrétaire peut annuler un rendez-vous existant. Le créneau horaire redevient disponible.
- **Consultation de l'agenda:**Le médecin consulte son agenda pour voir ses rendez-vous. Le patient consulte ses propres rendez-vous. Le secrétaire consulte tous les agendas.
- **Gestion de l'historique médical:**Lorsqu'un rendez-vous est créé ou confirmé, le système envoie automatiquement une notification au patient par email et/ou SMS.
- **Envoi de notifications:**Tout utilisateur doit s'authentifier avec un identifiant et un mot de passe pour accéder au système. L'utilisateur peut se déconnecter du système pour sécuriser son compte.

2.3 Besoins non fonctionnels:

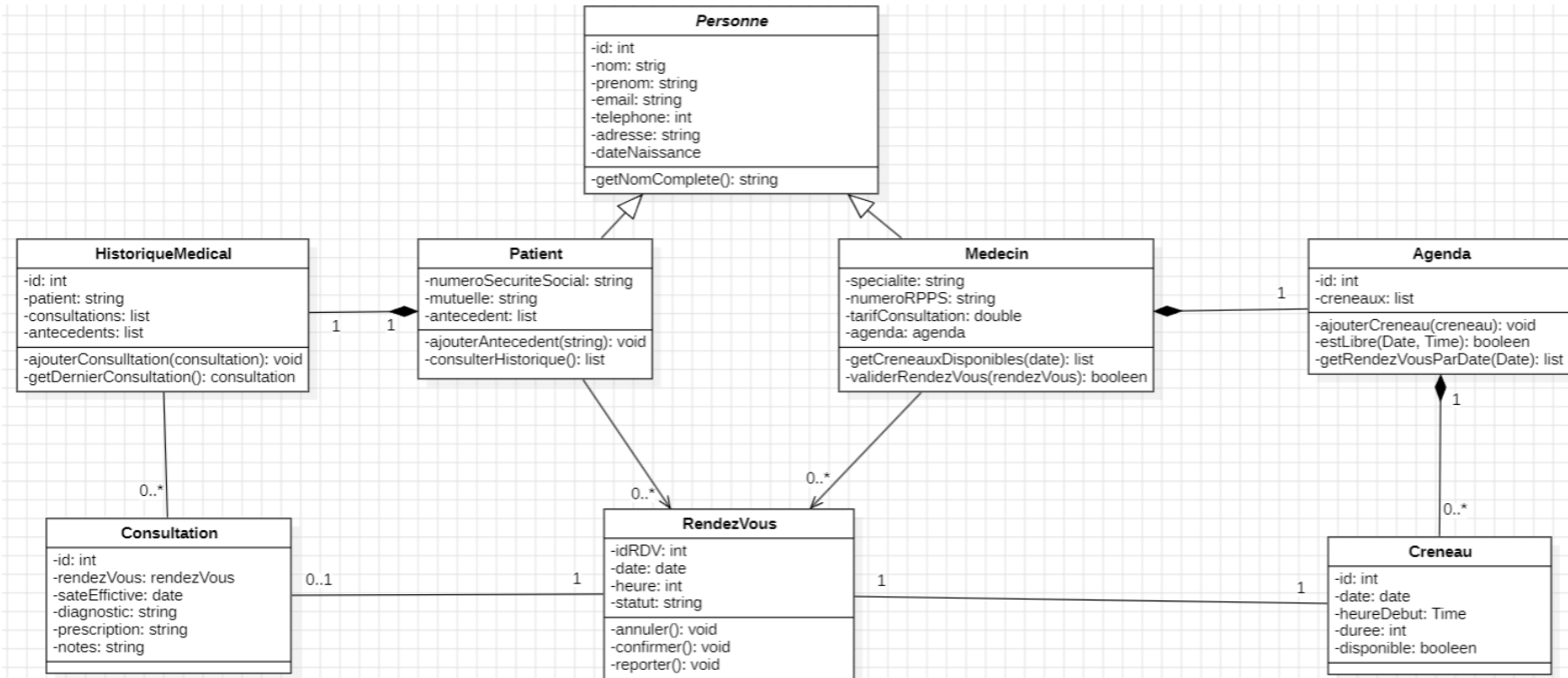
- **Sécurité :** Authentification obligatoire, accès différencié selon les rôles.
- **Disponibilité :** 24h/24 pour la prise de rendez-vous en ligne.
- **Performance :** Affichage de l'agenda en moins de 2 secondes.
- **Evolutivité :** Architecture permettant l'ajout de nouvelles fonctionnalités.

III. LES DIAGRAMMES UML:

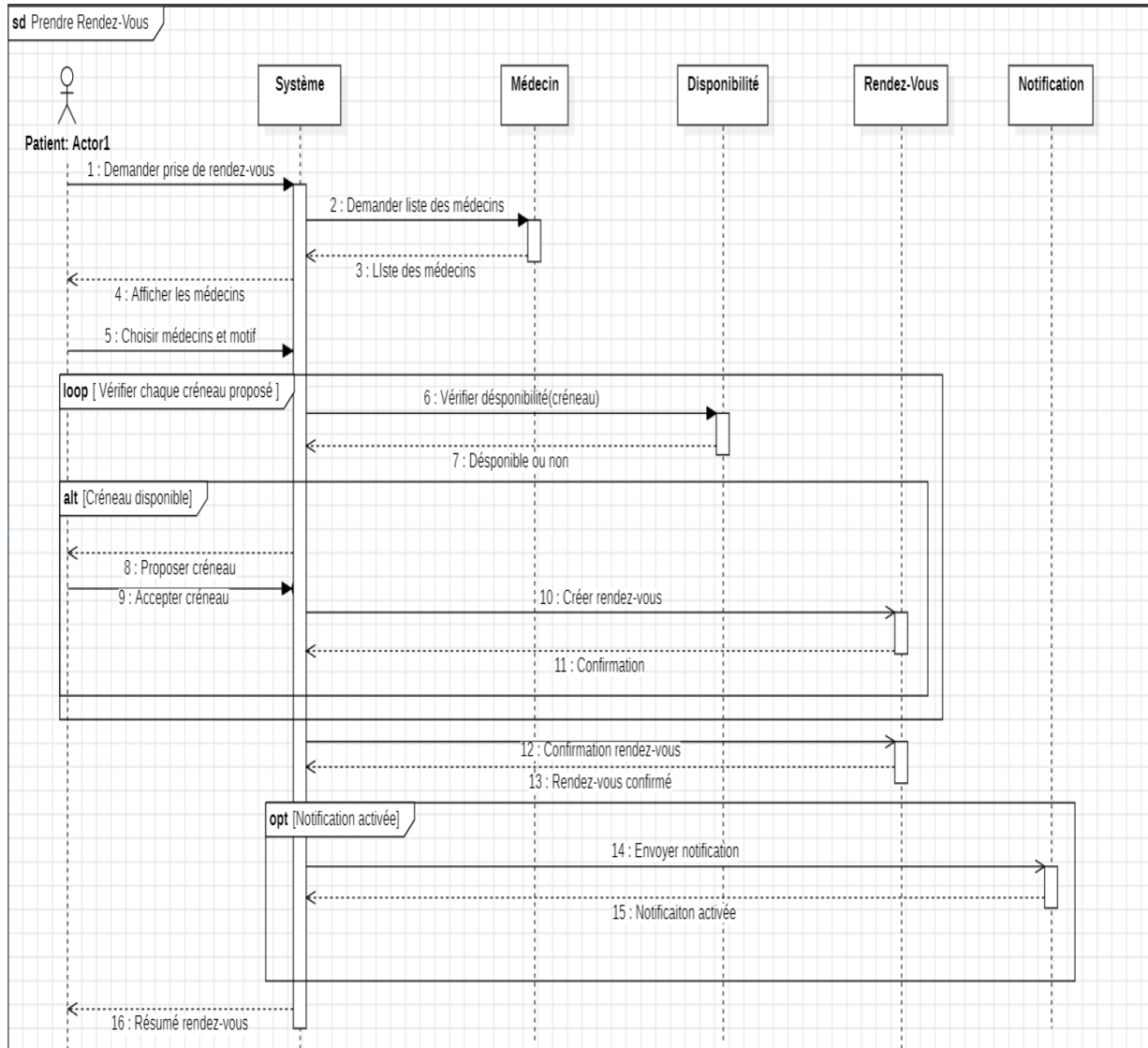
1. DIAGRAMME DE CAS D'UTILISATION:



2. DIAGRAMME DE CLASSES :



3.DIAGRAMME DE SEQUENCE:



Ce diagramme de séquence représente le processus de **prise de rendez-vous médical** entre un patient et le système.

Au début, le **patient** envoie une demande au **système** pour prendre un rendez-vous. Le système interagit ensuite avec le **médecin** afin de récupérer la liste des médecins disponibles, puis affiche cette liste au patient.

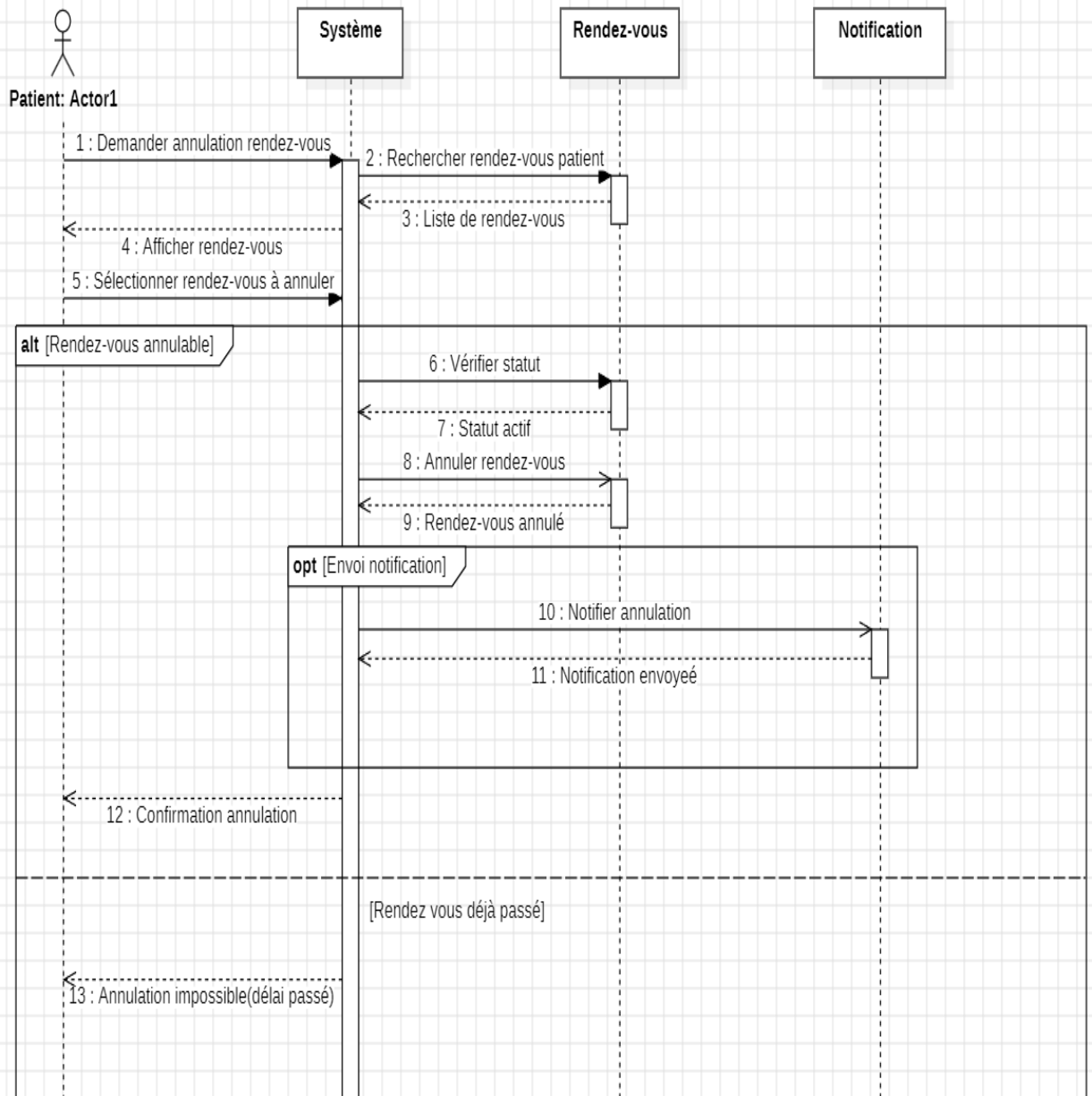
Le patient choisit un médecin ainsi que le motif de consultation. Ensuite, le système entre dans une **boucle de vérification des créneaux**. Pour chaque créneau proposé, il interroge le module de **disponibilité** pour vérifier si le créneau est libre.

> Si le créneau est **disponible**, le système le propose au patient. Si ce dernier l'accepte, un **rendez-vous** est créé et confirmé.

> Si le créneau est **indisponible**, le système informe le patient, qui doit proposer un autre créneau

Une fois un créneau validé, le système confirme définitivement le rendez-vous avec le module **RendezVous**. Ensuite, de manière optionnelle (bloc **opt**), une **notification** peut être envoyée au patient pour lui confirmer les détails du rendez-vous. Enfin, le système envoie au patient un **résumé du rendez-vous**.

sd Annuler rendez-vous



Ce diagramme de séquence décrit le processus d'annulation d'un rendez-vous médical par un patient.

Au début, le **patient** envoie une demande d'annulation au **système**. Le système interroge ensuite le module **RendezVous** pour récupérer la liste des rendez-vous associés au patient, puis les affiche.

Le patient sélectionne le rendez-vous qu'il souhaite annuler.

Ensuite, une structure conditionnelle (**alt**) est utilisée :

- Si le rendez-vous est **annulable (non encore passé)** :
 - Le système vérifie le statut du rendez-vous.
 - Si le statut est actif, le système procède à l'**annulation du rendez-vous**.
 - Une confirmation d'annulation est retournée au système.
 - De manière optionnelle (**opt**), une **notification** est envoyée au patient pour l'informer de l'annulation.
 - Enfin, le système affiche un message de **confirmation d'annulation** au patient.
- Si le rendez-vous est **déjà passé** :
 - Le système informe le patient que l'annulation est **impossible** en raison du dépassement du délai.

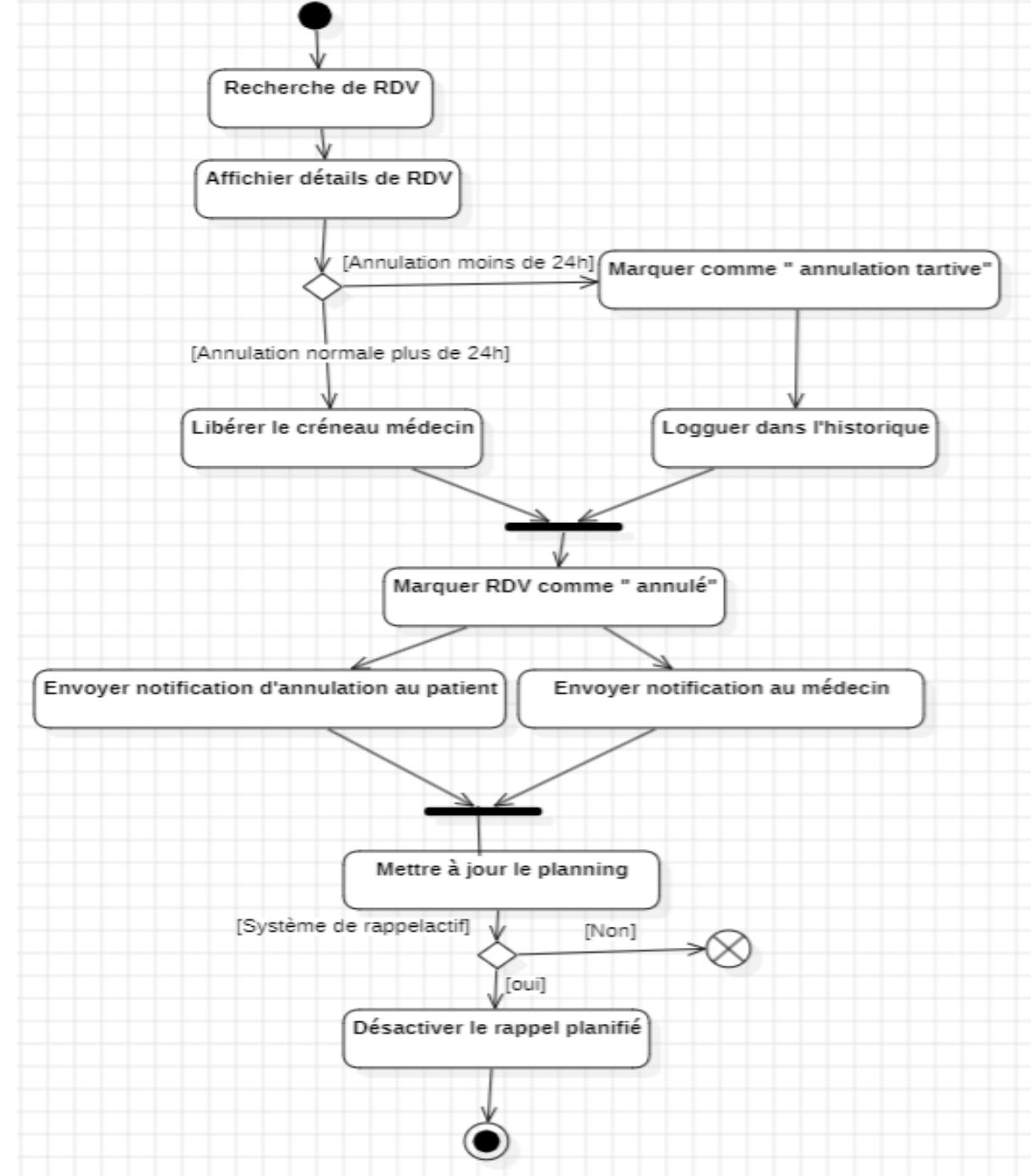
Le processus se termine par la réponse finale envoyée au patient.

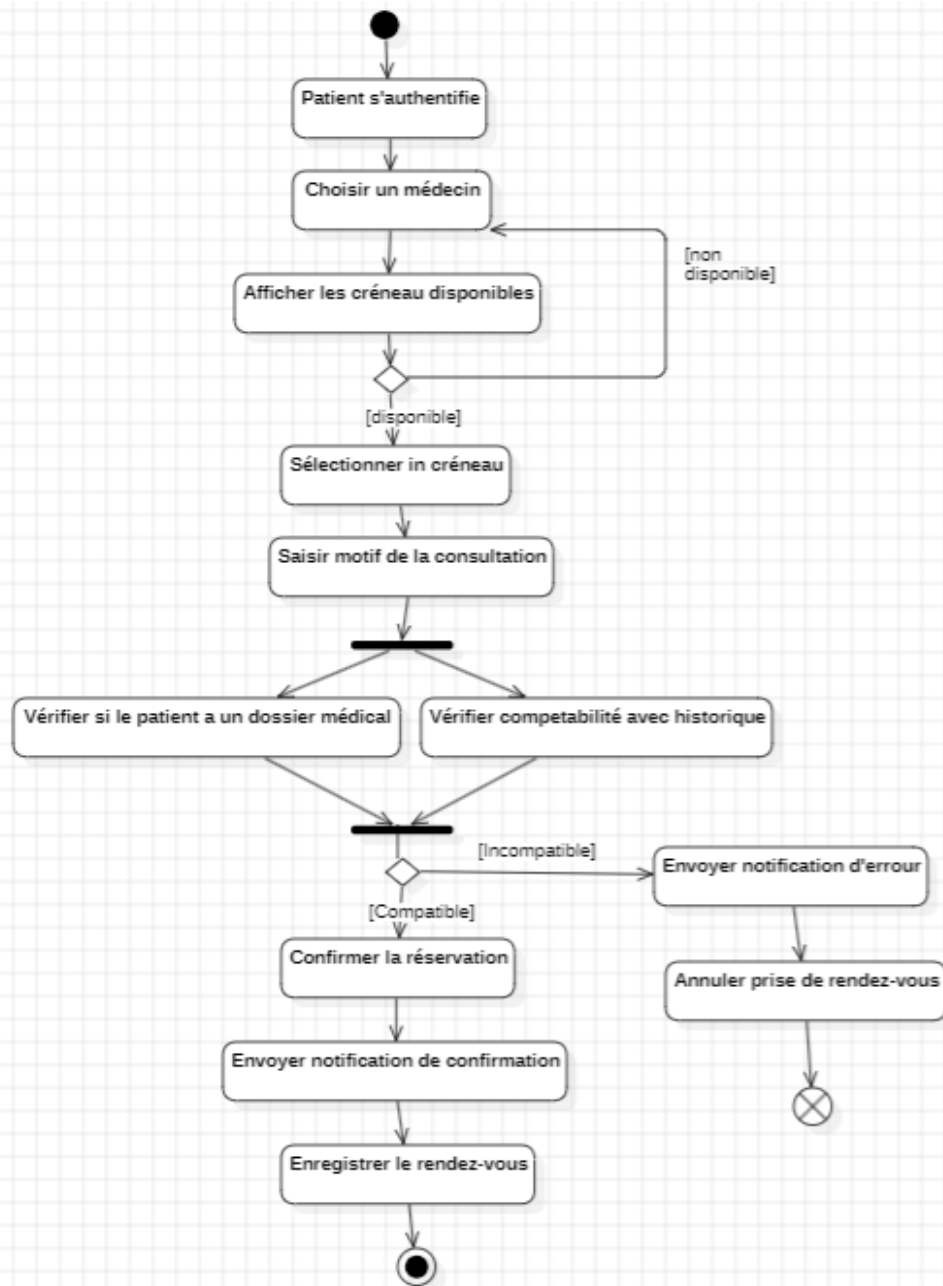
4. DIAGRAMME D'ACTIVITÉS :

1. « Prendre un rendez-vous médical »

Acteur principal : Patient

Objectif : Réserver un créneau disponible avec un médecin.



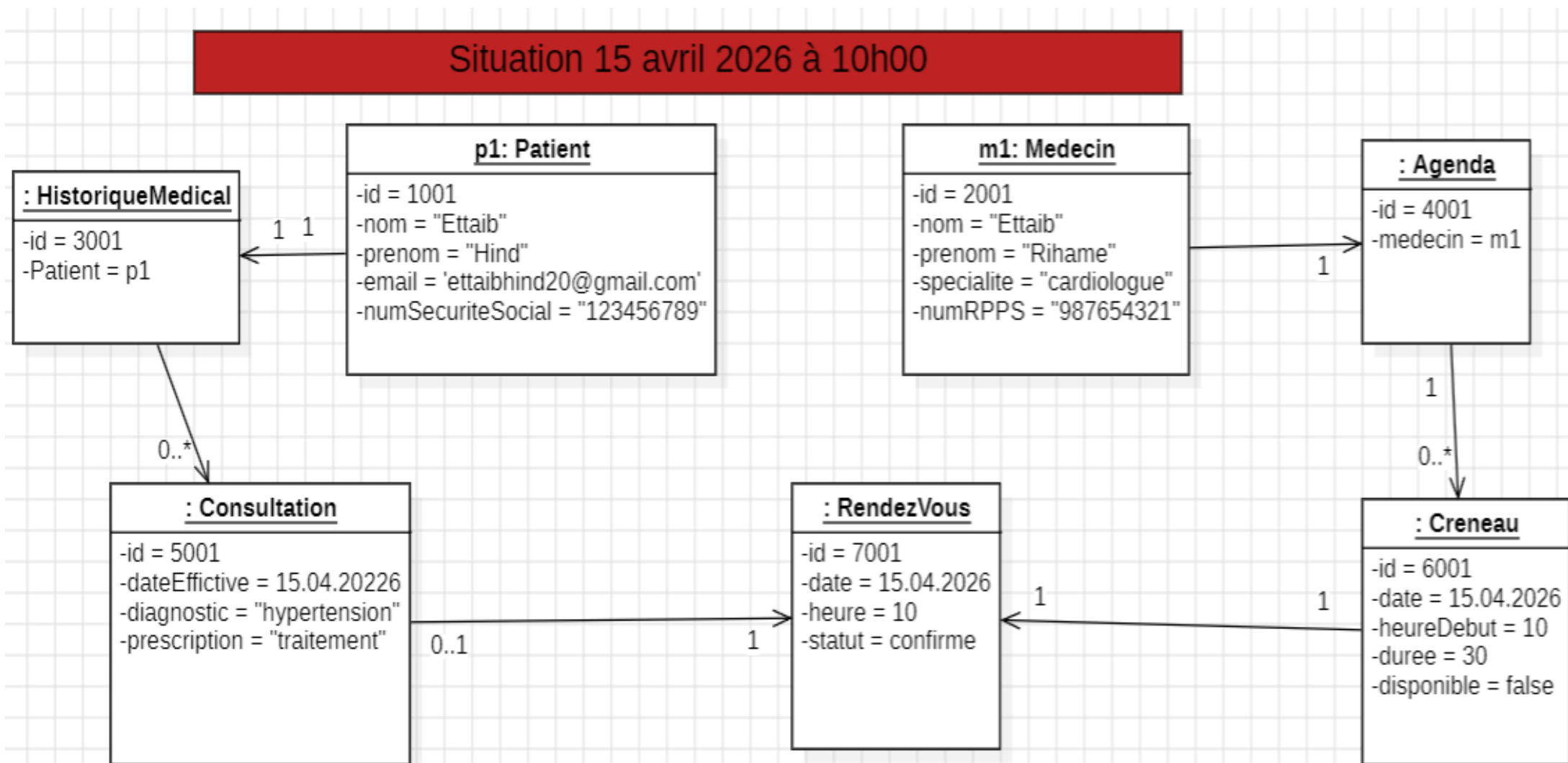


2. « Annuler un rendez-vous et notifier »

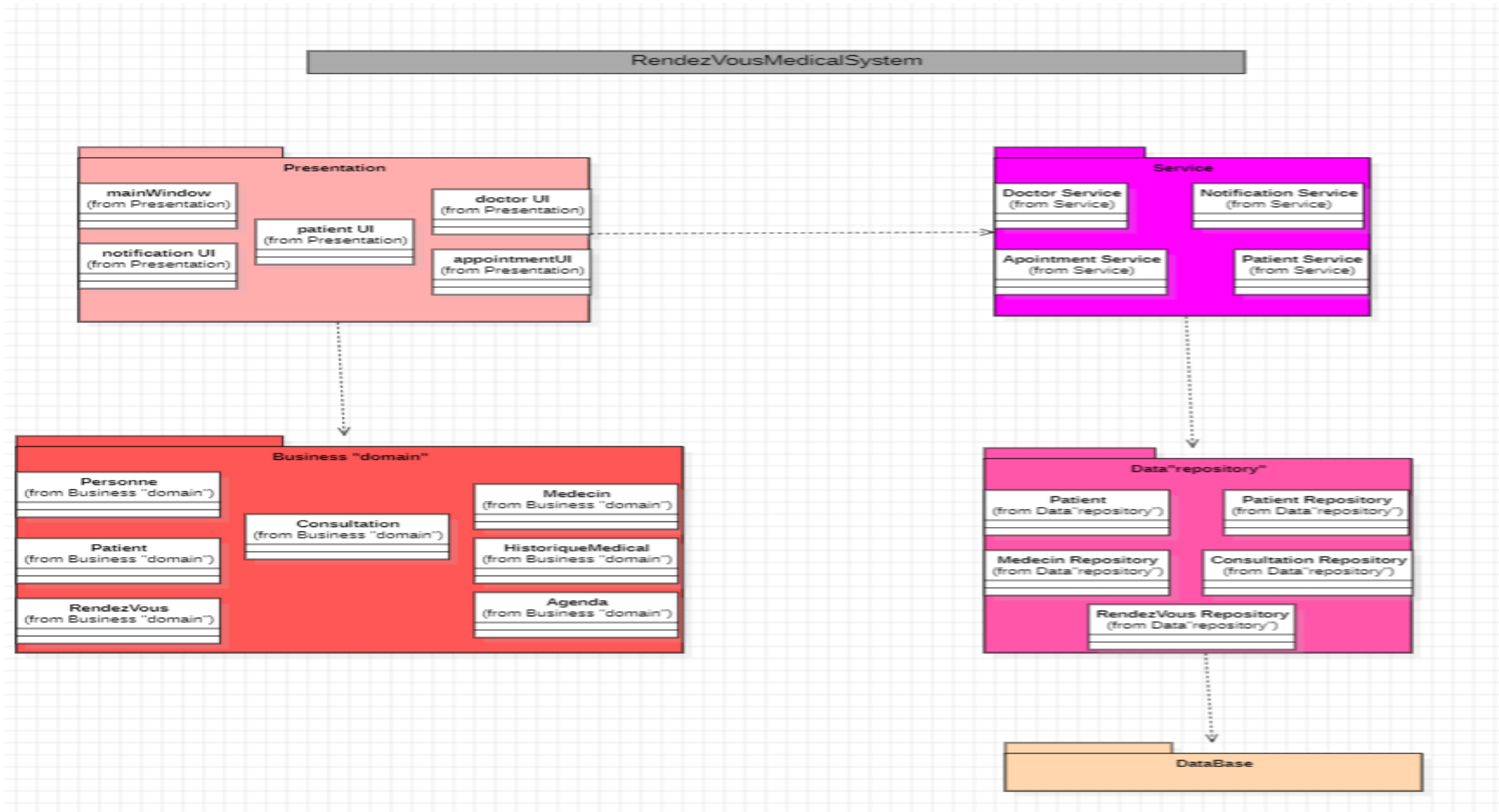
Acteur principal : Patient ou Secrétaire médicale

Objectif : Annuler un rendez-vous existant et libérer le créneau.

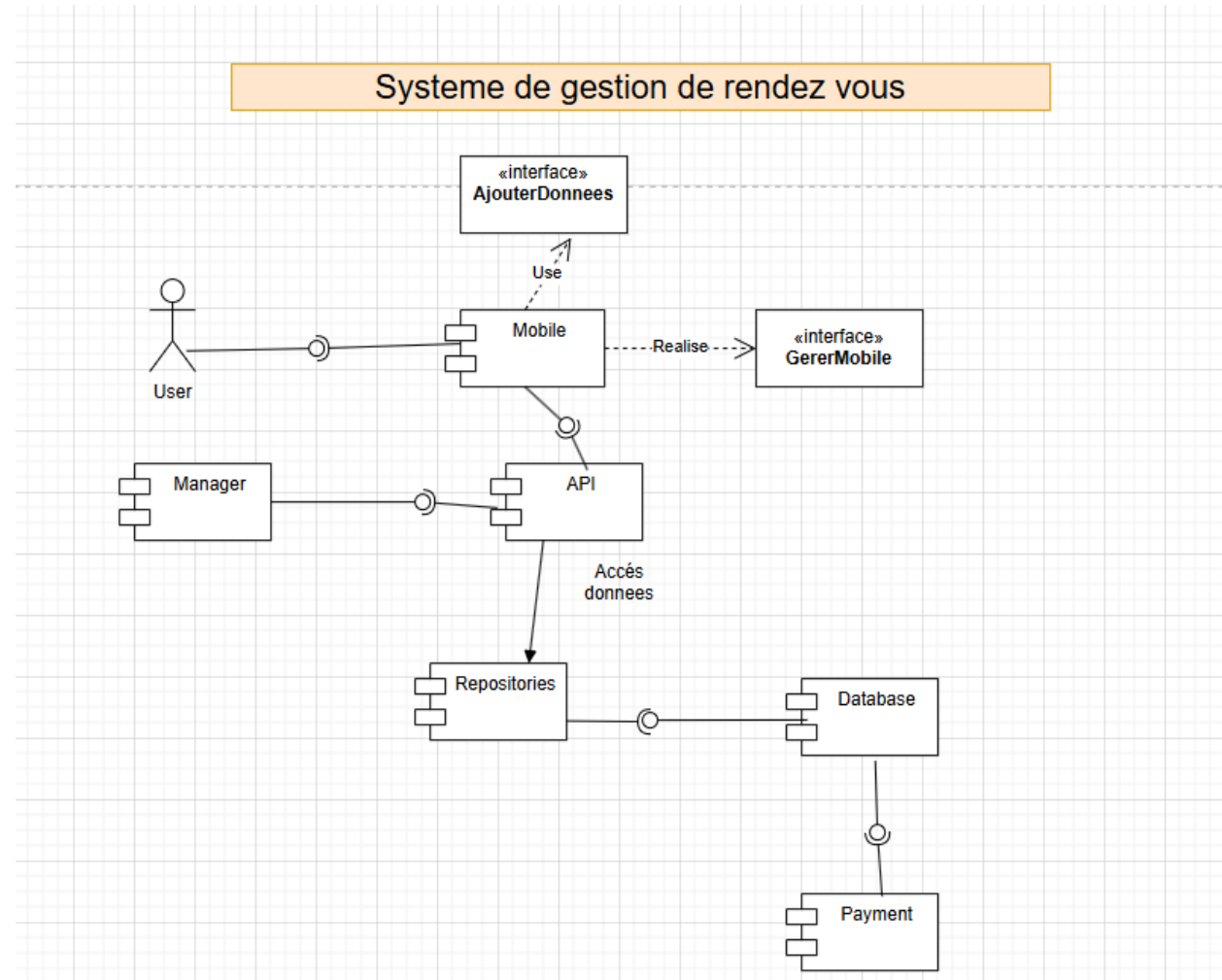
5. DIAGRAMME D'OBJECTS :



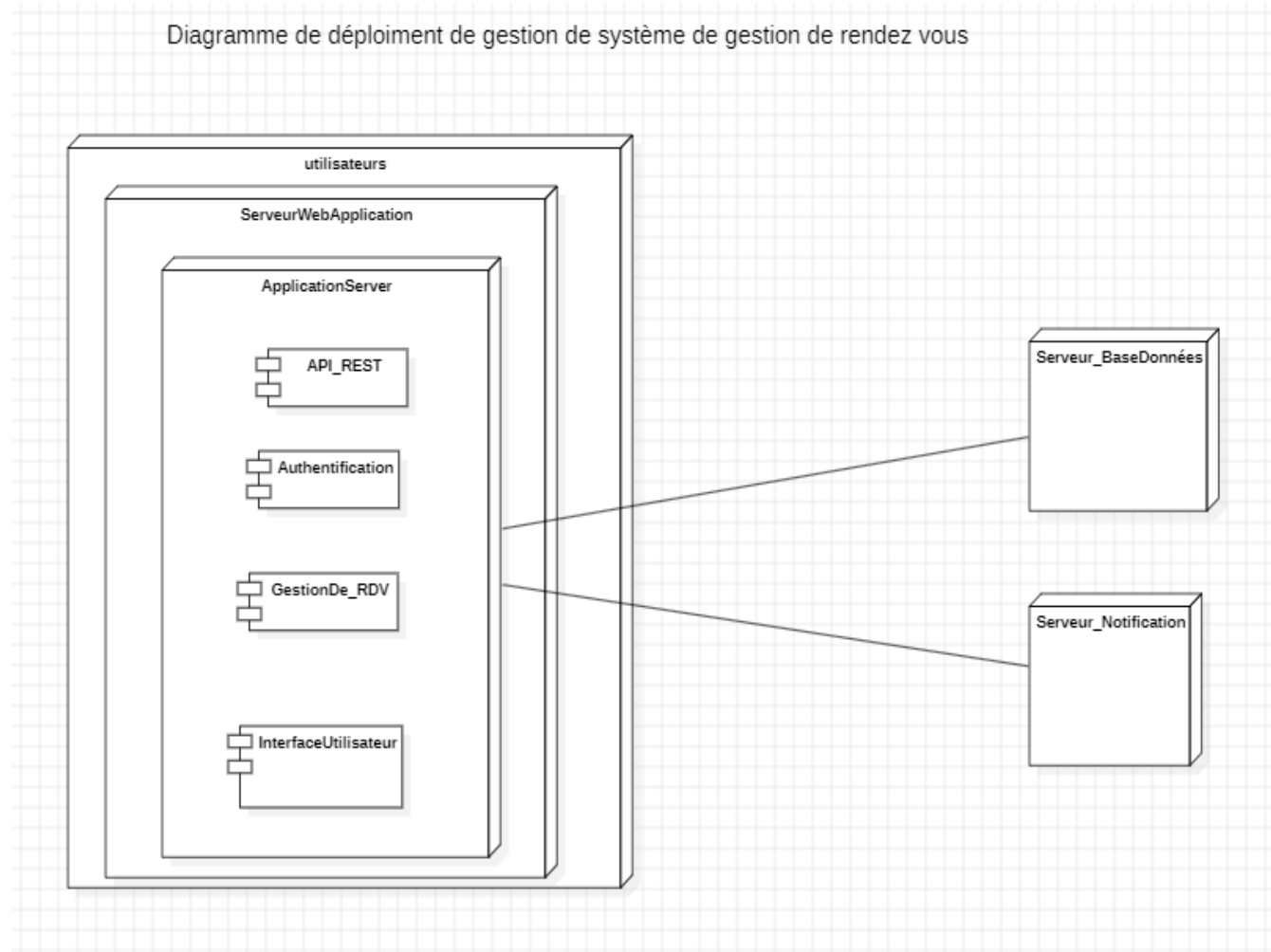
6:DIAGRAMME DE PAQUETAGES:



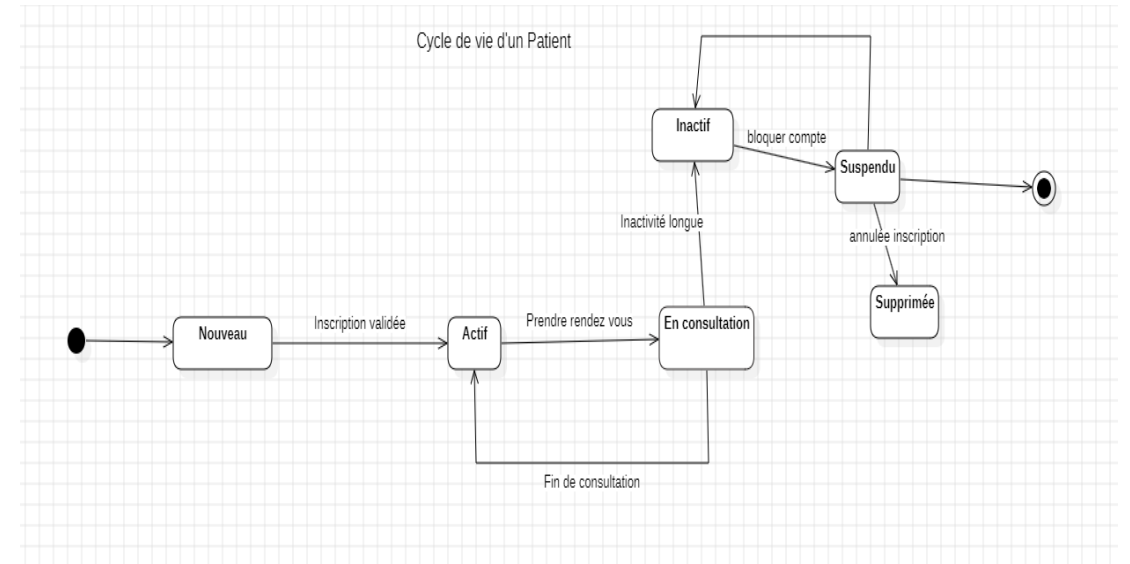
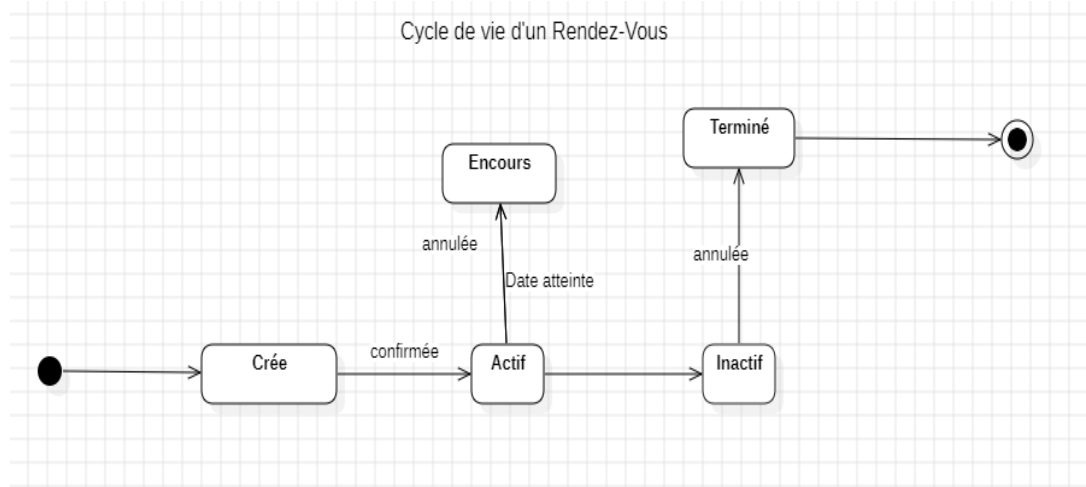
7. DIAGRAMME DE COMPOSANTS :



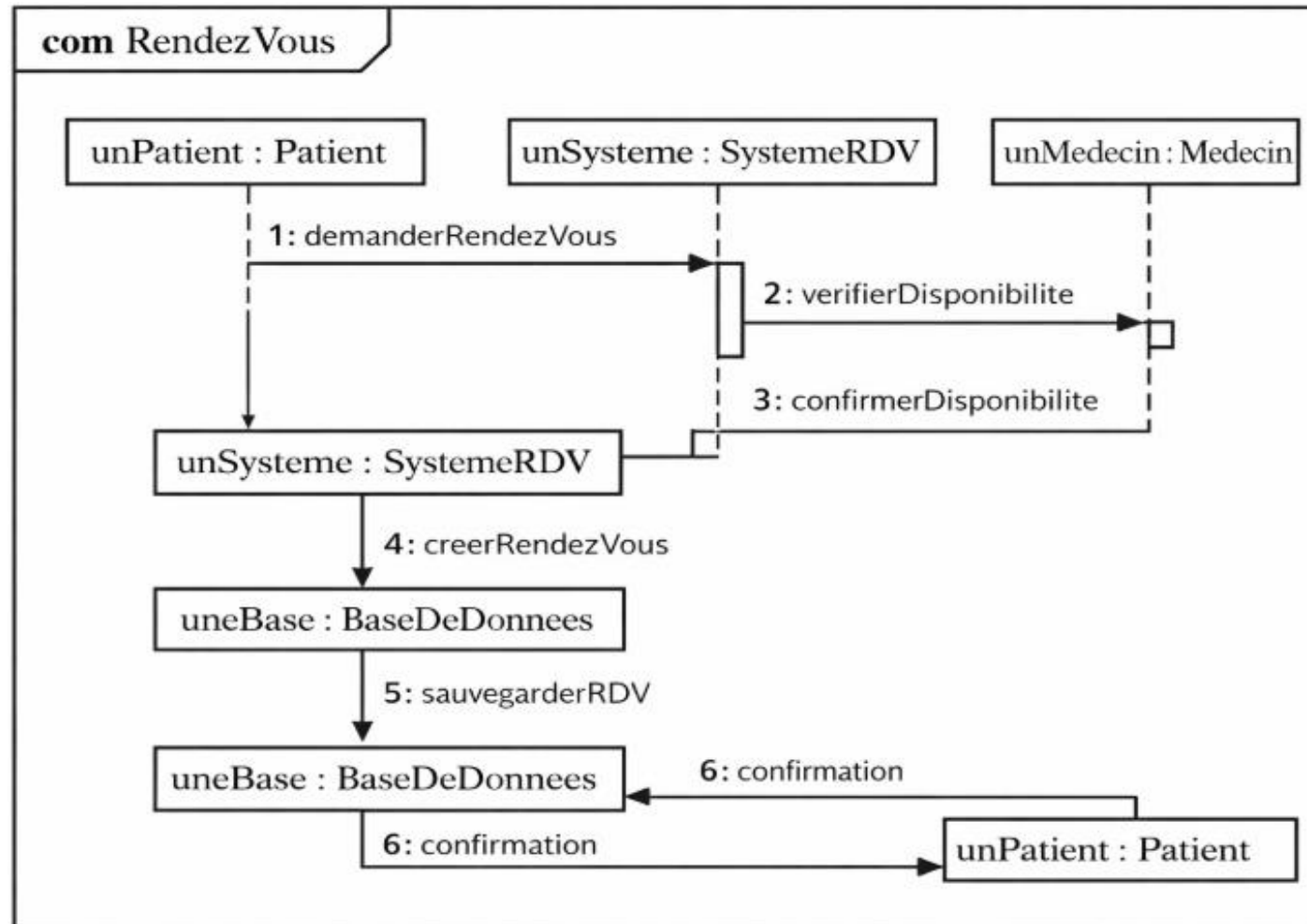
8. DIAGRAMME DE DEPLOIEMENTS :



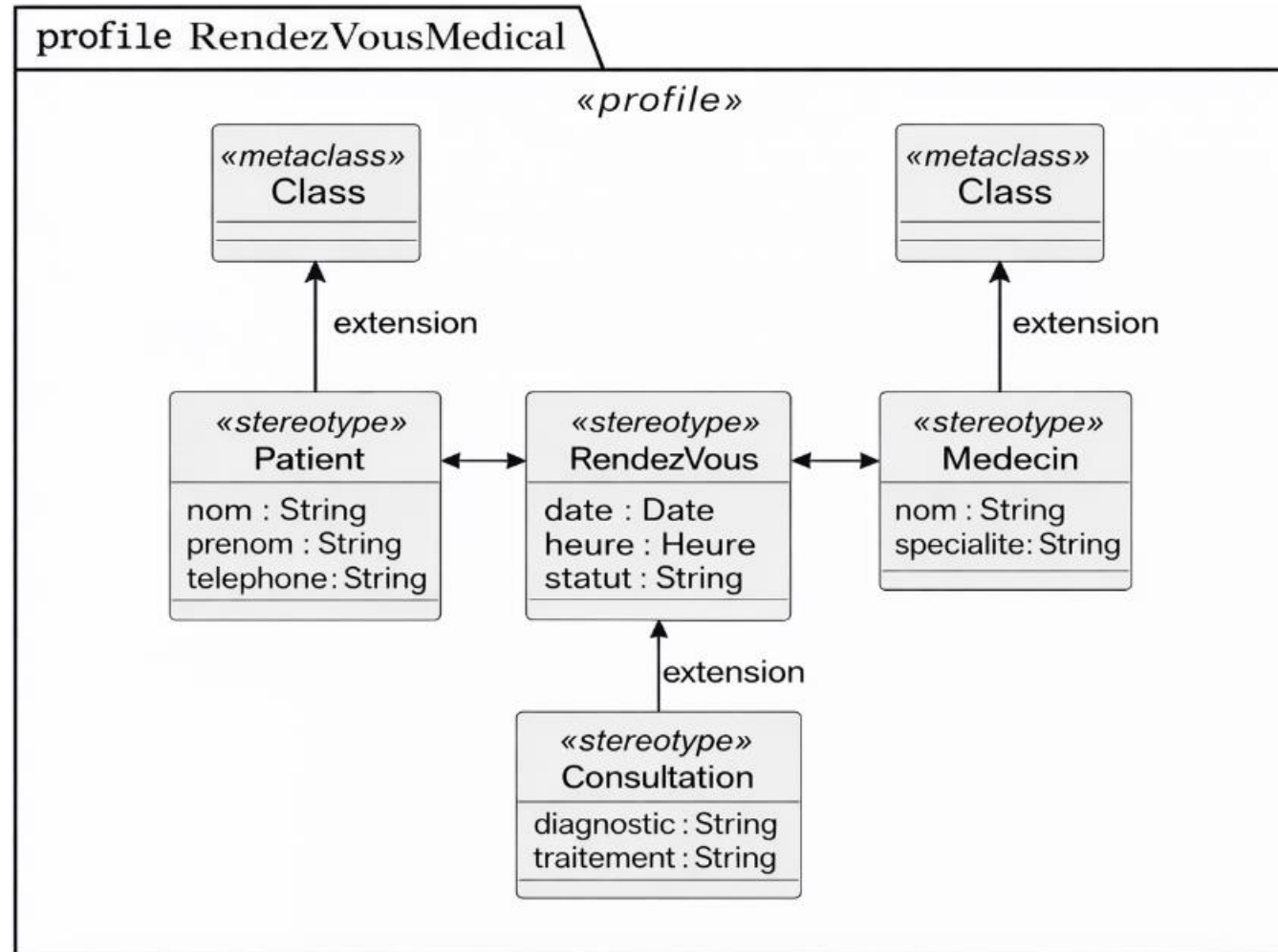
9. DIAGRAMME D'ETATS-TRANSITIONS:



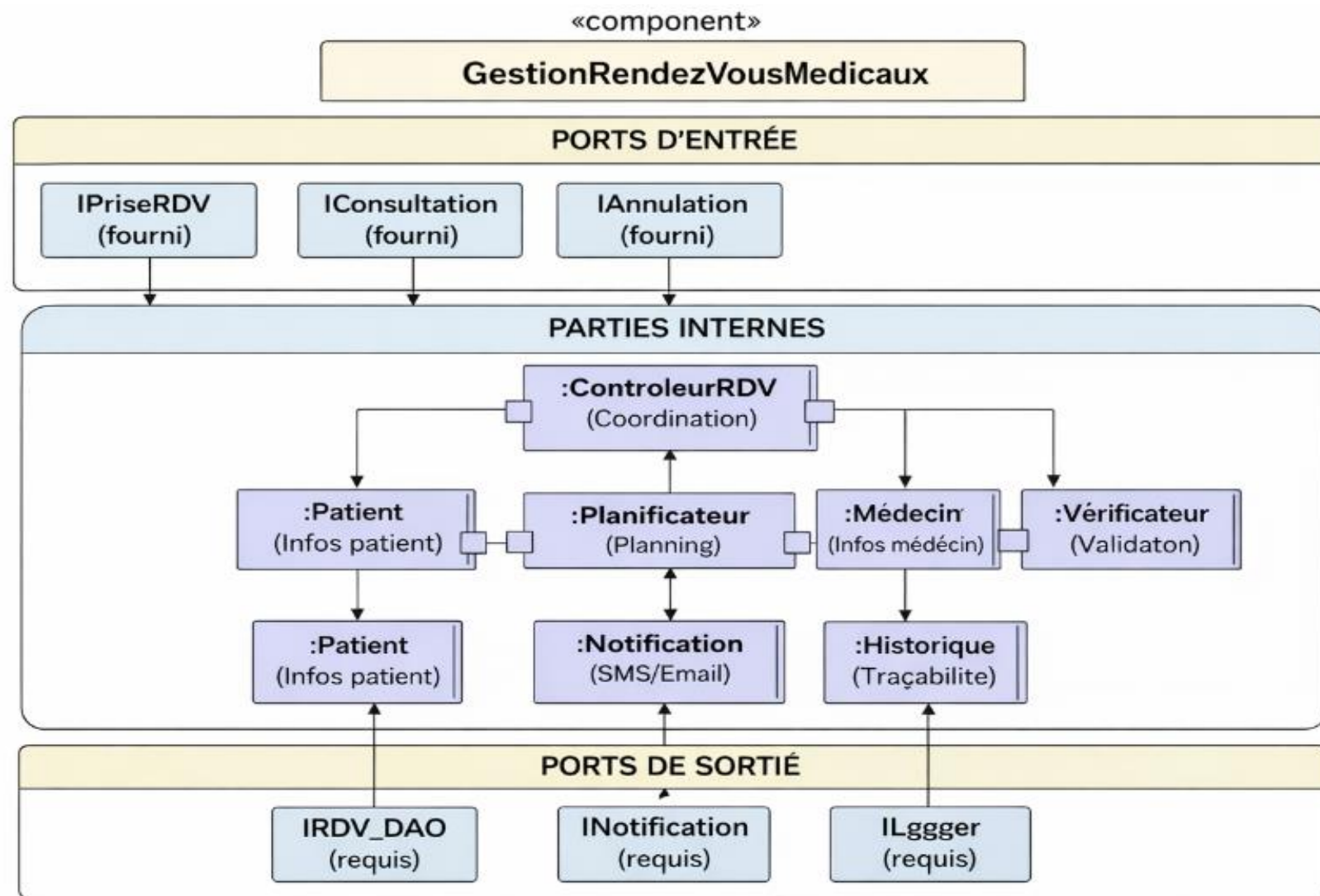
10.DIAGRAMME DE COMMUNICATION :



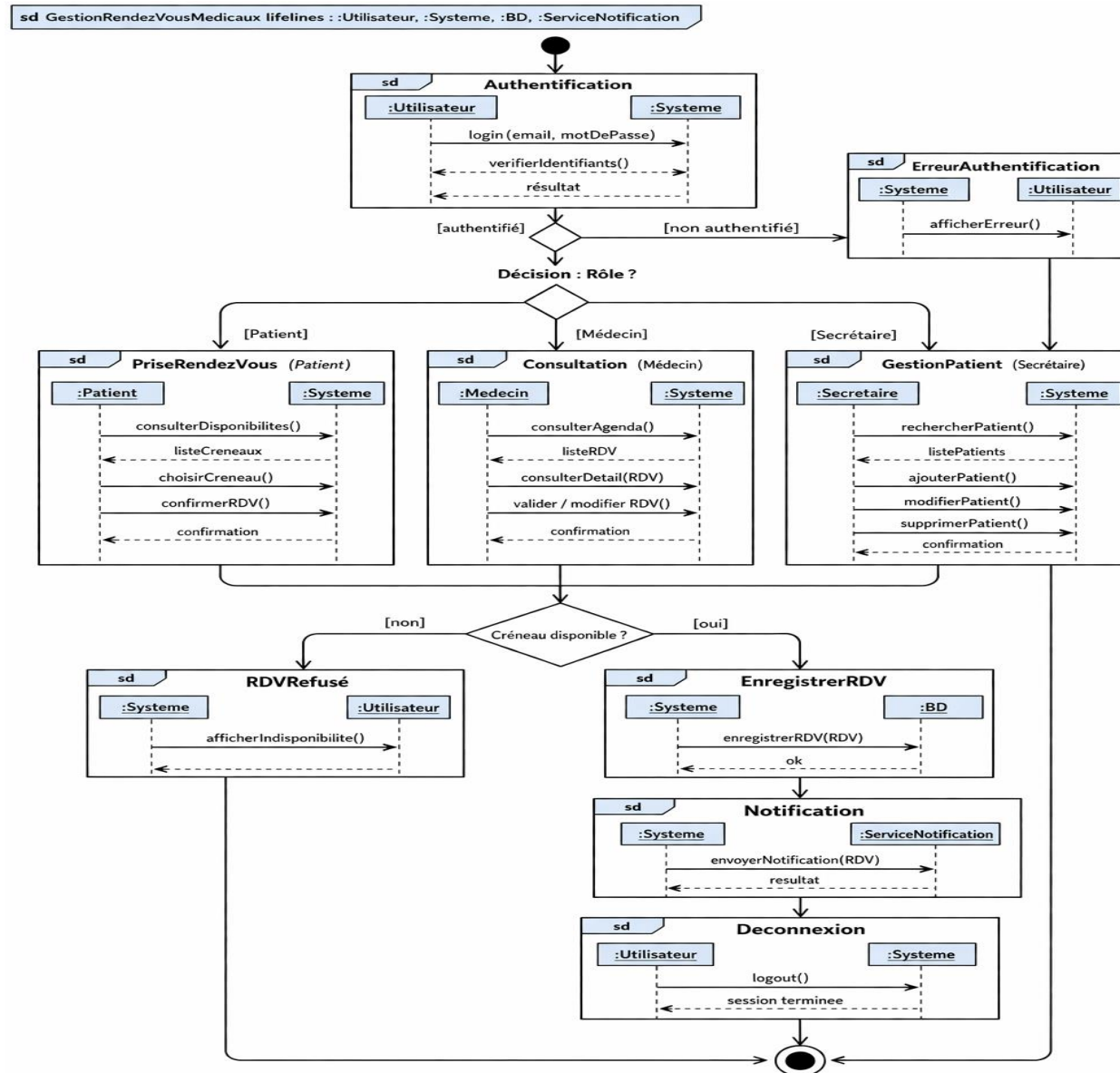
11.DIAGRAMME DE PROFIL :



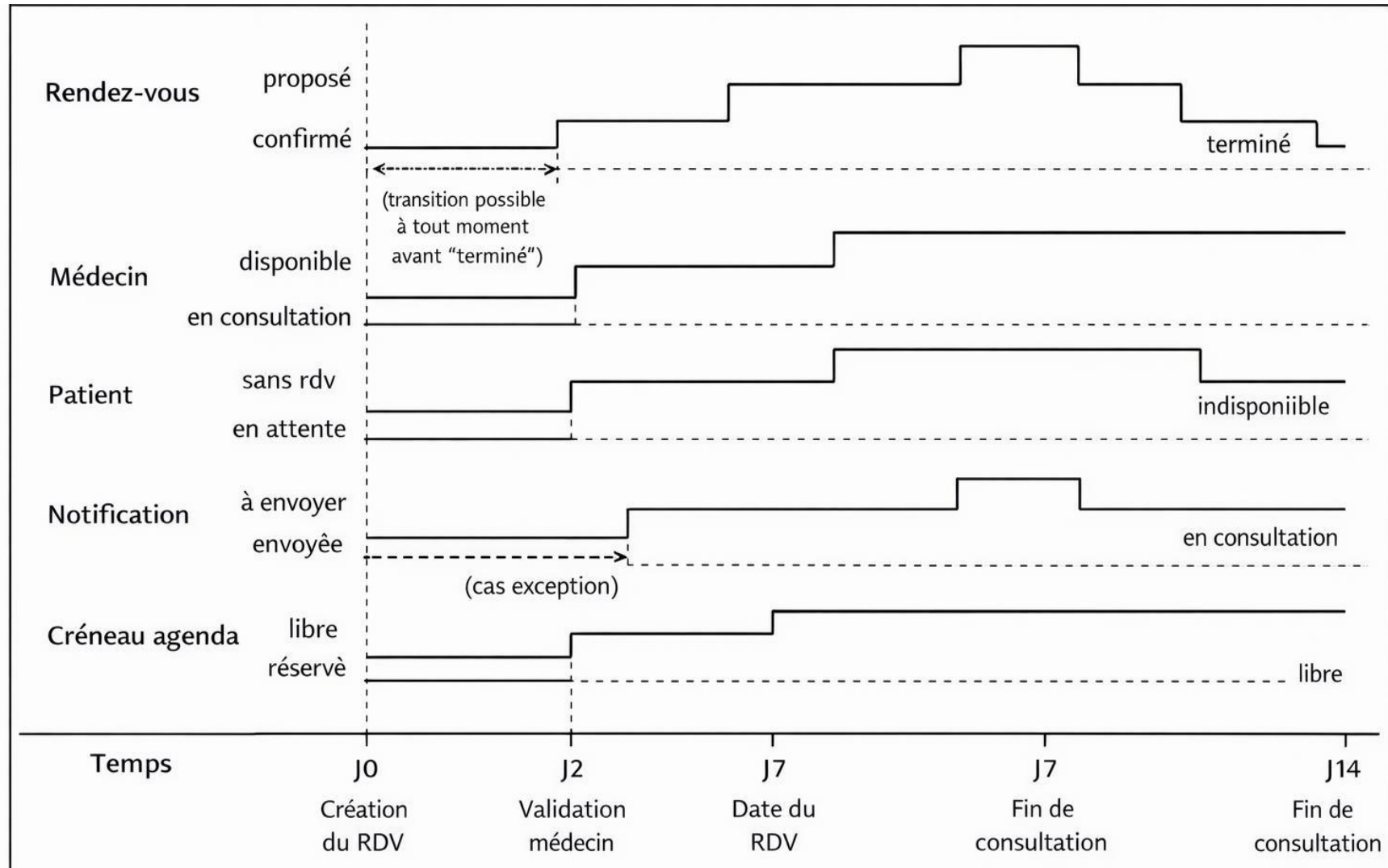
12.DIAGRAMME DE STRUCTURE DE COMPOSITE :



13. DIAGRAMME INTERACTION GLOBALE :



14.DIAGRAMME DE TEMPS :



IV. GENERATION DE CODE JAVA :

Bureau > Model1				
Trier ▾ Afficher ▾ ⋮				
Nom	Modifié le	Type	Taille	
Agenda.java	05/04/2026 10:56	Fichier source Java	1 Ko	
Consultation.java	04/04/2026 17:03	Fichier source Java	1 Ko	
Creneau.java	04/04/2026 17:03	Fichier source Java	1 Ko	
HistoriqueMedical.java	05/04/2026 10:26	Fichier source Java	1 Ko	
Medecin.java	05/04/2026 10:55	Fichier source Java	1 Ko	
Patient.java	04/04/2026 17:03	Fichier source Java	1 Ko	
Personne.java	04/04/2026 17:07	Fichier source Java	1 Ko	
RendezVous.java	05/04/2026 10:28	Fichier source Java	1 Ko	

Consultation.java

```
File Edit Selection View Go Run Terminal Help
J Consultation.java X J Personne.java J HistoriqueMedical.java J RendezVous.java J Patient.java J Medecin.java J Agenda.java J Creneau.java
C:\Users\hind>OneDrive\Desktop>Model1>J Consultation.java
1
2 import java.io.*;
3 import java.util.*;
4
5 /**
6  *
7  */
8 public class Consultation {
9
10     /**
11      * Default constructor
12      */
13     public Consultation() {
14     }
15
16     /**
17      *
18      */
19     private int id;
20
21     /**
22      *
23      */
24     private rendezVous rendezVous;
25
26     /**
27      *
28      */
29     private date sateEffective;
30
31     /**
32      *
33      */
34     private string diagnostic;
35
36     /**
37      *
38      */
39     private string notes;
40
41     /**
42      *
43      */
44     private string prescription;
45
46
47
48 }
```

Personne.java

```
C:\Users\hind>OneDrive\Desktop>Model1>J Personne.java >...
1 import java.io.*;
2 import java.util.*;
3
4 /**
5  *
6  */
7 public class Personne {
8
9     /**
10      * Default constructor
11      */
12     public Personne() {
13     }
14
15     /**
16      *
17      */
18     private int id;
19
20     /**
21      *
22      */
23     private string nom;
24
25     /**
26      *
27      */
28     private string prenom;
29
30     /**
31      *
32      */
33     private string email;
34
35     /**
36      *
37      */
38     private int telephone;
39
40     /**
41      *
42      */
43     private string adresse;
44
45     /**
46      *
47      */
48     public date dateNaissance;
49
50     /**
51      * @return
52      */
53     private string getNomComple() {
54         // TODO implement here
55         return null;
56     }
57 }
```

HistoriqueMedical.java

```
C:\> Users > hind > OneDrive > Desktop > Model1 > J HistoriqueMedical.java > ...
1  import java.io.*;
2  import java.util.*;
3
4  /**
5   *
6   */
7  public class HistoriqueMedical {
8
9      /**
10       * Default constructor
11       */
12     public HistoriqueMedical() {
13     }
14
15     /**
16      *
17      */
18     private int id;
19
20     /**
21      *
22      */
23     private String patient;
24
25     /**
26      *
27      */
28     private List consultations;
29
30     /**
31      *
32      */
33     private List antecedents;
34
35     /**
36      * @param consultation
37      * @return
38      */
39     private void ajouterConsultation(void consultation) {
40         // TODO implement here
41         return null;
42     }
43
44     /**
45      * @return
46      */
47     private Consultation getDernierConsultation() {
48         // TODO implement here
49         return null;
50     }
51
52 }
53
54 }
```

RendezVous.java

```
C:\> Users > hind > OneDrive > Desktop > Model1 > J RendezVous.java > RendezVous
1  import java.io.*;
2  import java.util.*;
3
4  /**
5   *
6   */
7  public class RendezVous {
8
9      /**
10       * Default constructor
11       */
12     public RendezVous() {
13     }
14
15     /**
16      *
17      */
18     private int idRDV;
19
20     /**
21      *
22      */
23     private Date date;
24
25     /**
26      *
27      */
28     private int heure;
29
30     /**
31      *
32      */
33     private String statut;
34
35     /**
36      * @return
37      */
38     private void annuler() {
39         // TODO implement here
40         return null;
41     }
42
43     /**
44      * @return
45      */
46     private void confirmer() {
47         // TODO implement here
48         return null;
49     }
50
51     /**
52      * @return
53      */
54     private void reporter() {
55         // TODO implement here
56         return null;
57     }
58
59 }
```


Medecin.java

```
C:\Users\hind> OneDrive > Desktop > Model1 > J Medecin.java > ...
1  import java.io.*;
2  import java.util.*;
3
4  /**
5   *
6   */
7  public class Medecin extends Personne {
8
9      /**
10       * Default constructor
11       */
12     public Medecin() {
13     }
14
15     /**
16      *
17      */
18     private String specialite;
19
20     /**
21      *
22      */
23     private String numeroRPPS;
24
25     /**
26      *
27      */
28     private double tarifConsultation;
29
30     /**
31      *
32      */
33     private agenda agenda;
34
35
36
37     /**
38      * @param date
39      * @return
40      */
41     private List getCreneauxDisponibles(void date) {
42         // TODO implement here
43         return null;
44     }
45
46     /**
47      * @param rendezVous
48      * @return
49      */
50     private boolean validerRendezVous(void rendezVous) {
51         // TODO implement here
52         return null;
53     }
54
55 }
```

Agenda.java

```
C:\Users\hind> OneDrive > Desktop > Model1 > J Agenda.java > ...
1  import java.io.*;
2  import java.util.*;
3
4  /**
5   *
6   */
7  public class Agenda {
8
9      /**
10       * Default constructor
11       */
12     public Agenda() {
13     }
14
15     /**
16      *
17      */
18     private int id;
19
20     /**
21      *
22      */
23     private List creneaux;
24
25
26
27     /**
28      * @param creneau
29      * @return
30      */
31     private void ajouterCreneau(void creneau) {
32         // TODO implement here
33         return null;
34     }
35
36     /**
37      * @param Date
38      * @param Time
39      * @return
40      */
41     private boolean estLibre(void Date, void Time) {
42         // TODO implement here
43         return null;
44     }
45
46     /**
47      * @param Date
48      * @return
49      */
50     private List getRendezVousParDate(void Date) {
51         // TODO implement here
52         return null;
53     }
54
55 }
```

Patient.java

```
C:\Users> hind > OneDrive > Desktop > Model1 > Patient.java > ...
1
2 import java.io.*;
3 import java.util.*;
4
5 /**
6  *
7  */
8 public class Patient extends Personne {
9
10     /**
11      * Default constructor
12      */
13     public Patient() {
14     }
15
16     /**
17      *
18      */
19     private String numeroSecuriteSocial;
20
21     /**
22      *
23      */
24     private String mutuelle;
25
26     /**
27      *
28      */
29     private List antecedent;
30
31
32
33     /**
34      * @param String
35      * @return
36      */
37     private void ajouterAntecedent(void String) {
38         // TODO implement here
39         return null;
40     }
41
42     /**
43      * @return
44      */
45     private List consulterHistorique() {
46         // TODO implement here
47         return null;
48     }
49
50 }
```

Creneau.java

```
C:\Users> hind > OneDrive > Desktop > Model1 > Creneau.java > ...
1
2 import java.io.*;
3 import java.util.*;
4
5 /**
6  *
7  */
8 public class Creneau {
9
10     /**
11      * Default constructor
12      */
13     public Creneau() {
14     }
15
16     /**
17      *
18      */
19     private int id;
20
21     /**
22      *
23      */
24     private Date date;
25
26     /**
27      *
28      */
29     private Time heureDebut;
30
31     /**
32      *
33      */
34     private int duree;
35
36     /**
37      *
38      */
39     private boolean disponible;
40
41
42
43 }
```

V. CONCLUSION :

- Dans ce cahier des charges, nous avons étudié et modélisé le système de gestion des rendez-vous médicaux en utilisant les diagrammes UML.
- Cette étude nous a permis de mieux comprendre le fonctionnement global du système, les interactions entre les différents acteurs (patient, médecin, système), ainsi que les différents états du rendez-vous (proposé, confirmé, en cours, terminé ou annulé).
- Les différents diagrammes réalisés (cas d'utilisation, diagramme de classes, diagramme de séquence, diagramme d'états et diagramme de déploiement) ont facilité la visualisation et l'organisation des fonctionnalités du système.
- Ce travail constitue une base solide pour la phase de développement, en assurant une meilleure structuration du projet, une réduction des erreurs, et une communication claire entre les membres de l'équipe.
- En conclusion, l'utilisation d'UML dans ce projet a permis de concevoir un système cohérent, évolutif et adapté aux besoins des utilisateurs.